

Contents

Preface	11
1 Advanced Robotics Applied to Drones	19
1.1 Module Summary	21
1.2 What is ROS2?	22
1.2.1 History and Evolution	22
1.2.2 Why ROS2 for Drones?	22
1.2.3 ROS2 Versions	23
1.3 Installing ROS2 Humble on Ubuntu 22.04	24
1.3.1 Prerequisites	24
1.3.2 Step 1: Configure Ubuntu Repository	24
1.3.3 Step 2: Install ROS2 Humble	25
1.3.4 Step 3: Configure the Environment	25
1.3.5 Step 4: Verify Installation	26
1.4 ROS2 Fundamental Concepts	28
1.4.1 ROS2 Architecture	28
1.4.2 Key Concepts: Nodes, Topics, Services	29
1.5 Creating Your First ROS2 Node in Python	33
1.5.1 ROS2 Package Structure	33
1.5.2 Example 1: Simple Publisher Node	34
1.5.3 Example 2: Subscriber Node	36
1.6 Ardupilot – ROS2 Integration	37
1.6.1 How Does Ardupilot Communicate with ROS2?	37
1.6.2 Installing MAVROS	37
1.6.3 Example: Drone Control via ROS2	38
1.6.4 Complete Package: FlightStatus	41
1.7 Autonomous Navigation with Nav2	45

1.7.1	What is Nav2?	45
1.7.2	Installing Nav2	46
1.7.3	Example: Navigate to Point of Interest (POI)	46
1.8	Building and Running Packages	49
1.8.1	Build Package	49
1.8.2	Run Node	49
1.9	Essential ROS2 Tools	50
1.9.1	ros2 topic: Explore Topics	50
1.9.2	ros2 service: Services	51
1.9.3	ros2 node: Nodes	51
1.10	Simulation: Ardupilot + Gazebo + ROS2	52
1.10.1	Why Simulate?	52
1.10.2	Installing SITL + Gazebo	52
1.10.3	Example: SITL + ROS2 Simulation	52
1.11	Visualisation with RViz	53
1.11.1	Launch RViz	53
1.11.2	Visualise Drone	53
1.12	Sensor Integration in ROS2	54
1.12.1	GPS Sensor	54
1.12.2	IMU Sensor (Inertial Measurement Unit)	56
1.13	Real-World Use Cases	58
1.13.1	Autonomous Aerial Mapping	58
1.13.2	Object Tracking	58
1.13.3	Drone Swarm	59
1.14	Debugging and Troubleshooting	60
1.14.1	Common Issues	60
1.15	Best Practices and Design Patterns	63
1.15.1	Naming Nodes and Topics	63
1.15.2	Professional Logging	64
1.15.3	Robust Error Handling	65
1.15.4	Using Configuration Parameters	66
1.16	Intensive Practical Exercises	68
1.16.1	Exercise 1: ROS2 Chat	68
1.16.2	Exercise 2: Telemetry Monitor	68
1.16.3	Exercise 3: Interactive Navigator	68
1.17	Code Repository	69
1.18	Resources and References	70
1.18.1	Official Documentation	70

1.18.2	Community	70
1.19	Assessment	71
1.19.1	Knowledge Quiz	71
1.19.2	Practical Exercise	73
1.19.3	Integration Project	74
1.19.4	Assessment Solutions	76
1.20	Final Notes and Next Steps	79
1.20.1	Current Limitations	79
1.20.2	Advanced Paths	79
1.20.3	Final Tips	79
1.21	Glossary	80
2	Computer Vision Applied to Drones	83
2.1	Module Summary	85
2.2	Foundations of Computer Vision	87
2.2.1	What is Computer Vision?	87
2.2.2	Typical Vision Pipeline in Drones	88
2.2.3	Vision Computing: CPU vs GPU vs TPU	89
2.3	OpenCV: Fundamentals	90
2.3.1	Installation and Setup	90
2.3.2	Video Capture	90
2.3.3	Basic Image Processing	92
2.4	Object Detection: From Classical to Deep Learning	94
2.4.1	Classical Methods vs Deep Learning	94
2.5	YOLO v8: Real-Time Object Detection	95
2.5.1	Why is YOLO the Standard in Drones?	95
2.5.2	Installing YOLOv8	96
2.5.3	Example 1: Basic Detection	97
2.5.4	Example 2: Real-Time Detection from Camera	98
2.5.5	Example 3: Object Tracking	100
2.5.6	Available YOLO v8 Models	101
2.6	Alternative Technologies (Overview)	103
2.7	YOLO + ROS2 Integration	104
2.8	Optimization for Embedded Hardware	106
2.8.1	Converting to TensorRT	106
2.9	Use Cases in Drones	107
2.9.1	Person Detection + Alert	108
2.9.2	Obstacle Avoidance	108

2.9.3	Real-Time Detection Flow	109
2.10	Intensive Practical Exercises	110
2.10.1	Exercise 5.1: HSV Color Detector	110
2.10.2	Exercise 5.2: YOLO Detection	110
2.10.3	Exercise 5.3: Persistent Tracking	110
2.11	Best Practices	111
2.12	Next Step: Module 6 — Artificial Intelligence Applied to Drones	112
2.13	Additional Resources	113
2.14	Assessment	114
2.14.1	Knowledge Quiz	114
2.14.2	Practical Exercise: Real-Time HSV Color Detector	115
2.14.3	Integrating Project: Complete Vision System in ROS2	116
2.14.4	Assessment Solutions	117
2.15	Glossary of Terms	119
3	Artificial Intelligence Applied to Drones	123
3.1	Module Summary	125
3.2	The Fundamental Problem: Latency in Drones	127
3.2.1	Why is 100ms the Critical Limit?	127
3.2.2	Latency Breakdown in Drones	129
3.3	Why Cloud AI DOES NOT Work in Drones?	130
3.3.1	The Cloud Computing Illusion	130
3.3.2	Additional Cloud AI Problems	131
3.4	Jetson: NVIDIA Platforms for Edge AI	132
3.4.1	What is Jetson?	132
3.4.2	Jetson Product Line	133
3.4.3	Jetson SDK Installation	134
3.5	Online vs Offline Inference	135
3.5.1	Online Inference (During Flight)	136
3.5.2	Offline Inference (Post-Mission)	138
3.6	Power Optimization in Jetson	140
3.6.1	YOLO Power Consumption on Jetson	140
3.6.2	Power Optimization Strategies	141
3.7	Integration: ROS2 + Jetson + Ardupilot	143
3.7.1	Full System Architecture	143
3.7.2	Complete Example: Obstacle-Avoiding Drone	145

3.8	Practical Use Cases: Embedded AI in Drones	148
3.8.1	1. Autonomous Search and Rescue	148
3.8.2	2. Infrastructure Inspection	148
3.8.3	3. Precision Agriculture	148
3.8.4	4. Aerial Mapping with 3D Analysis	149
3.9	Step-by-Step Deployment: Your First AI Drone	150
3.9.1	Step 1: Prepare Jetson	150
3.9.2	Step 2: Create ROS2 Package with YOLO	151
3.9.3	Step 3: Build and Run	151
3.10	Troubleshooting and Optimization	152
3.10.1	Problem: YOLO is Too Slow	152
3.10.2	Problem: Jetson Overheats and Throttles	152
3.10.3	Problem: Battery Drains Quickly	152
3.11	Practical Exercises	153
3.11.1	Exercise 6.1: Measure Real Latency	153
3.11.2	Exercise 6.2: Convert Model to TensorRT	154
3.11.3	Exercise 6.2b (Optional): Power Monitoring on Jetson	155
3.11.4	Exercise 6.3: Person-Following Drone	156
3.11.5	Exercise 6.4: Post-Mission Analysis	157
3.12	Additional Resources	159
3.13	Assessment	160
3.13.1	Knowledge Quiz	160
3.13.2	Practical Exercise: Real Latency Measurement	161
3.13.3	Integration Project: Autonomous Drone with Embedded AI	162
3.13.4	Assessment Solutions	163
3.14	Conclusion: The Future of AI-Powered Drones	166
3.15	Glossary of Terms	167
A	Git for Drones	171
A.1	Introduction	173
A.2	Git Installation	174
A.2.1	Windows	174
A.2.2	macOS	174
A.2.3	Linux (Ubuntu/Debian)	174
A.3	Initial Configuration	175
A.4	Cloning Your First Repository	176

A.4.1	Step 1: Find the Repository URL	176
A.4.2	Step 2: Open Terminal / Command Prompt	177
A.4.3	Step 3: Create a Folder for the Courses	177
A.4.4	Step 4: Clone the Repository	178
A.5	Structure of the Downloaded Project	179
A.6	Detailed Practical Examples	180
A.6.1	Example 1: Download Volume 1 Code (Hardware, Ardupilot, MAVLink)	180
A.6.2	Example 2: Download Code from Both Volumes	181
A.6.3	Example 3: Update Existing Code	182
A.7	Updating the Code	183
A.8	Essential Commands (Cheat Sheet)	184
A.9	Assessment	185
A.9.1	Knowledge Quiz	185
A.9.2	Practical Exercise	187
A.9.3	Integration Project	188
A.9.4	Assessment Solutions	190
A.10	Glossary	192
A.10.1	Essential Git Terms	192
A.10.2	Platform Terms	193
A.10.3	Quick Git Commands (Summary)	193
A.10.4	Project Educational Terms	194
B	C++ for Drones	195
B.1	Introduction	197
B.2	Installing ROS2 and Compilers	198
B.2.1	ROS2 Humble (Ubuntu 22.04)	198
B.2.2	Compilers	198
B.3	Basic C++ Theory	199
B.3.1	Data Types	199
B.3.2	Conditionals: if, else, switch	200
B.3.3	Loops: for and while	201
B.3.4	Functions	202
B.4	Structure of a ROS2 Package in C++	204
B.5	Downloading Course Examples	205
B.5.1	Option 1: On Linux and macOS (Recommended)	205
B.5.2	Option 2: On Windows with Git Bash	205
B.5.3	Option 3: Download ZIP directly	206

B.6 Your First Node: Publisher	207
B.6.1 Complete Code	207
B.6.2 Code Explanation	208
B.6.3 What Happens When You Run It	208
B.6.4 Adapting for Drone Applications	208
B.7 Compilation with CMake	209
B.8 Build and Run	210
B.9 Publisher vs Subscriber	211
B.10 Advantages of C++ in Drones	212
B.11 Assessment	213
B.11.1 Knowledge Quiz	213
B.11.2 Practical Exercise	215
B.11.3 Integration Project	216
B.11.4 Assessment Solutions	218
B.12 Glossary of Terms	220
B.12.1 C++ Terms	220
B.12.2 ROS2 Terms	220
B.12.3 Build Terms	221
B.12.4 Drone Application Terms	221
Glossary	223
References	225